INSTITUTE OF ENGINEERING AND MANAGEMENT
Advanced Programming (OOP)

Mental Wellness Tracker

STUDENT NAME	**Ahona Sarkar**
ROLL NUMBER	70
ENROLLMENT NUMBER	12014052019062
INSTRUCTORS	Dr. Swagatam Basu, Dr. Susovan Jana, Dr. Soumadip Biswas
SEMESTER	Spring 2026.

2. Problem Statement

University students and working professionals often fail to recognise early signs of burnout and mental fatigue. Subtle patterns — poor sleep, overwork, rising stress — go unnoticed until they become serious.

EmpathAI addresses this by providing a Java-based console application that logs daily behavioural signals — sleep duration, work hours, and self-reported stress levels — and generates weekly wellness reports with risk classifications and actionable recommendations.

The project demonstrates eight core Object-Oriented Programming concepts within a layered architecture that is easy to understand and extend. It models real-world behavioural data through a clean inheritance hierarchy, polymorphic scoring, interface-based reporting, and robust file persistence.

Key Features

Daily Signal Logging Track sleep hours, work hours, and self-reported stress (1-10) for any date.	**Stress Scoring Engine** Rule-based algorithm classifies each day as LOW, MODERATE, or HIGH risk.
Weekly Reports Visual score bars, risk summaries, and personalised recommendations.	**File Persistence** Save and load multiple user profiles across sessions with robust error handling.

3. System Design

3.1 UML Class Diagram

```
«interface»
┌──────────────────┐
│    Reportable    │
├──────────────────┤
│+generateReport(User)│
│+generateReport(U,S) │
└────────┬──────────┘
         │ implements
┌────────┴──────────┐  ┌──────────────────┐
│ ReportGenerator   │──│ BehaviorAnalyzer  │
├───────────────────┤  ├──────────────────┤
│-analyzer (HAS-A)  │  │+analyzeDay()     │
│+generateReport()  │  │+analyzeWeek()    │
└───────────────────┘  └──────────────────┘

«abstract»
┌──────────────────────────────┐
│      BehaviorSignal          │
├──────────────────────────────┤
│ -signalType: String          │
│ -date: String                │
├──────────────────────────────┤
│ +getStressContribution(): int  │ ◄ abstract
│ +getSummary(): String          │ ◄ abstract
│ +toString(): String            │
└──────────┬───────────────────┘
       │ extends (IS-A)
  ┌────────┴──────────────┐
  ▼        ▼        ▼
SleepSignal  WorkSession  StressEntry
-hoursSlept  -hoursWorked  -selfRating
             │ HAS-A (List<BehaviorSignal>)
       ┌────────┴────────┐
       │  DailyRecord    │
       │-date            │
       │+getTotalScore()│
       │+getRiskLevel()  │
       └────────┬────────┘
            │ HAS-A (List<DailyRecord>)
       ┌────────┴────────┐
       │     User        │
       │-userId, -name   │
       │+addRecord()     │
       └────────┬────────┘
            │ USES
  ┌──────────┴──────────┐  ┌──────────┐
  │    MainApp          │  │FileManager│
  │+main()              │──│+saveUser()│
  │-users: HashMap      │  │+loadUser()│
  └─────────────────────┘  └──────────┘
```

3.2 Class Descriptions

Class	Type	Responsibility
BehaviorSignal	Abstract Class	Base for all signals. Defines abstract getStressContribution() and getSummary().
SleepSignal	Concrete (extends BehaviorSignal)	Tracks hours slept. Scores +2 for <6hrs, +4 for <4hrs.
WorkSession	Concrete (extends BehaviorSignal)	Tracks hours worked. Scores +2 for >8hrs, +4 for >12hrs.
StressEntry	Concrete (extends BehaviorSignal)	Self-rated stress 1-10. Scores +2 for >=6, +4 for >=8.
DailyRecord	Model (Composition)	Container for one day. Aggregates signals. Computes risk level.
User	Model (Encapsulation)	User profile with all DailyRecords. Private fields, getters only.
BehaviorAnalyzer	Service Layer	analyzeDay() and analyzeWeek() — pattern detection and recommendations.
ReportGenerator	Implements Reportable	Formats weekly and daily wellness reports with score bars.
FileManager	I/O Layer	Serializes User data to tagged text. Loads and reconstructs on demand.
MainApp	Entry Point	10-option console menu. Input validation helpers. Top-level error handling.

3.3 Interaction Flow

1. Application starts from MainApp.main() — banner and menu displayed
2. User creates a profile or loads one from file via FileManager
3. Daily records are added: SleepSignal, WorkSession, StressEntry created and attached to DailyRecord
4. DailyRecord.getTotalStressScore() loops over signals — polymorphism resolves each subclass
5. BehaviorAnalyzer computes daily and weekly patterns
6. ReportGenerator formats output with ASCII score bars and recommendations
7. FileManager serializes User data to data/<userId>.txt using try-with-resources
8. On next launch, data is fully reconstructed from file — sessions are persistent

4. OOP Concepts Demonstrated

Concept	Where Applied	Class(es)
Abstraction	abstract class with abstract methods	BehaviorSignal
Inheritance	extends keyword, IS-A hierarchy	SleepSignal, WorkSession, StressEntry
Polymorphism	getStressContribution() via base type	DailyRecord.getTotalStressScore()
Encapsulation	private fields, getters only, no setters	User, DailyRecord
Composition	HAS-A with List<>, not IS-A	User > DailyRecord > BehaviorSignal
Interface	implements Reportable contract	ReportGenerator
Method Overloading	same name, different parameters	generateReport() x3
Exception Handling	try-with-resources, typed throws, re-prompt	FileManager, MainApp

4.1 Abstraction

BehaviorSignal — abstract class

Where: BehaviorSignal.java
Why: Hides implementation. Forces all subclasses to define their own scoring and summary logic.

```
public abstract class BehaviorSignal {
    public abstract int getStressContribution();
    public abstract String getSummary();
    // toString() is concrete — shared by all subclasses
    public String toString() { return "[" + signalType + "] " + getSummary(); }
}
```

4.2 Inheritance

SleepSignal, WorkSession, StressEntry

Where: All three extend BehaviorSignal
Why: Reduces redundancy. Common fields (signalType, date) and toString() are inherited. Only scoring logic differs.

```
public class SleepSignal extends BehaviorSignal {
    public SleepSignal(String date, double hoursSlept) {
        super("SLEEP", date);   // calls BehaviorSignal constructor
        this.hoursSlept = hoursSlept;
    }
    @Override
    public int getStressContribution() {
        if (hoursSlept < 4) return 4;
        if (hoursSlept < 6) return 2;
        return 0;
    }
}
```

4.3 Polymorphism

DailyRecord.getTotalStressScore()

Where: DailyRecord.java — loop over List<BehaviorSignal>
Why: Same method name resolves to different logic at runtime. No instanceof checks needed. Pure dynamic dispatch.

```
public int getTotalStressScore() {
    int total = 0;
    for (BehaviorSignal signal : signals) {
        total += signal.getStressContribution(); // polymorphic call
    }        // Java picks SleepSignal / WorkSession / StressEntry at runtime
    return total;
}
```

4.4 Encapsulation

User — all fields private, no public setters

Where: User.java
Why: Protects data integrity. Name and age can only be set via constructor. getRecords() returns a defensive copy.

```
public class User {
    private String userId;    // can only be read, never changed externally
    private String name;
    private int age;
    private List<DailyRecord> records = new ArrayList<>();

    public String getName()            { return name; }
    public List<DailyRecord> getRecords(){ return new ArrayList<>(records); }
    // NO setName(), setAge() — mutation through addRecord() only
}
```

4.5 Composition (HAS-A)

User HAS DailyRecord, DailyRecord HAS BehaviorSignal

Where: User.java, DailyRecord.java
Why: Models ownership — if User is deleted, records go with it. This is more correct than inheritance.

```
User
 └─ List<DailyRecord>        // User HAS records
     └─ List<BehaviorSignal>   // DailyRecord HAS signals

ReportGenerator
 └─ BehaviorAnalyzer          // ReportGenerator HAS an analyzer
```

4.6 Interface

Reportable interface implemented by ReportGenerator

Where: ReportGenerator.java
Why: Decouples the contract from the implementation. A future PDFReportGenerator could implement the same interface.

```
interface Reportable {
    String generateReport(User user);
    String generateReport(User user, String specificDate);
}
```

```
public class ReportGenerator implements Reportable {
    @Override public String generateReport(User user) { ... }
    @Override public String generateReport(User u, String date) { ... }
}
```

4.7 Method Overloading

generateReport() — three versions with same name

Where: ReportGenerator.java
Why: Consistent API. Caller always calls generateReport(). Java picks the right version at compile time from the arguments.

```
String generateReport(User user)            // weekly — 7 records
String generateReport(User user, String date) // single day report
void   printReport(User user)               // prints directly to console
```

4.8 Exception Handling

Layered at three levels: model constructors, FileManager, MainApp

Where: Throughout all classes
Why: Prevents crashes at every entry point. try-with-resources guarantees stream closure. Re-prompting loops improve UX.

```
// Level 1 — Model validation
if (selfRating < 1 || selfRating > 10)
    throw new IllegalArgumentException("Stress rating must be 1-10");

// Level 2 — File I/O
try (BufferedWriter w = new BufferedWriter(new FileWriter(path))) {
    w.write(data);  // auto-closed even if exception thrown
}

// Level 3 — Input helper re-prompts instead of crashing
private static int readInt(String prompt) {
    while (true) {
        try { return Integer.parseInt(readLine(prompt)); }
        catch (NumberFormatException e) { System.out.println("Enter a number."); }
    }
}
```

5. Implementation Highlights

5.1 Stress Scoring Algorithm

Each signal subclass implements getStressContribution() independently. The thresholds reflect real-world wellness research — chronic sleep deprivation carries the highest weight.

Signal	Condition	Score	Justification
Sleep	< 4 hours	+4	Severe deprivation
Sleep	4 – 6 hours	+2	Insufficient rest
Sleep	> 9 hours	+1	Possible oversleeping
Work	> 12 hours	+4	Severe overwork
Work	8 – 12 hours	+2	Extended session
Work	6 – 8 hours	+1	Mild overtime
Stress	Rating 8–10	+4	High self-reported
Stress	Rating 6–7	+2	Moderate self-report
Stress	Rating 4–5	+1	Mild self-report

Day total: 0–3 = LOW | 4–7 = MODERATE | 8+ = HIGH

5.2 File Serialization Format

Data is saved as human-readable tagged text. Tags make parsing robust — unknown lines trigger a warning and are skipped gracefully rather than crashing the load.

```
USER:u001,Ahona Sarkar,20
RECORD:2025-03-27
SLEEP:4.5
WORK:10.0
STRESS:8
RECORD:2025-03-28
SLEEP:7.0
WORK:6.0
STRESS:3
```

5.3 Sample Report Output

The weekly report uses ASCII score bars so it reads clearly in any terminal or plain-text viewer.

```
User     : Ahona Sarkar
Total days tracked : 5
──────────────────────────────────────────────────
2025-03-25 ▒▒▒
```

```
        └─ [SLEEP]  Slept 4.5 hours (+2)
        └─ [WORK]   Worked 10.0 hours (+2)
        └─ [STRESS] Self-reported stress: 8/10 (+4)
2025-03-26 [MODERATE]  (3 signals)
2025-03-27 [HIGH]      (3 signals)
2025-03-28 [LOW]       (3 signals)
2025-03-29 [MODERATE]  (3 signals)
──────────────────────────────────────────────
Average score : 6.4 / 12
Recommendation: Monitor your patterns and prioritize rest.
```

6. Testing & Error Handling

6.1 Test Cases

Test Case	Input	Expected Result
Create valid user	ID, name, valid age	User created and set as active
Add daily record	Date, sleep, work, stress	Record saved with risk level shown
View weekly report	User with 5 records	Score bars + recommendation printed
Invalid stress rating	Stress = 11	IllegalArgumentException displayed
Invalid age	Age = -5	Validation error message shown
Non-numeric input	Text in hours field	Re-prompt, no crash
Load non-existent user	Unknown user ID	FileNotFoundException with message
Save and reload user	u001 with 3 records	Full data reconstructed from file
Corrupt file line	Unknown tag in data file	Warning printed, line skipped

6.2 Edge Cases Handled

- Empty input fields — readLine() loops until non-empty string provided
- Stress rating outside 1–10 — IllegalArgumentException from StressEntry constructor
- Negative or zero age — IllegalArgumentException from User constructor
- Non-numeric input for numeric fields — NumberFormatException caught, user re-prompted
- Loading a user ID with no saved file — FileNotFoundException with descriptive message
- Corrupt or unrecognised lines in data file — warning printed, line skipped gracefully
- Viewing report with no records — guard method requireRecords() shows friendly message
- Duplicate date entry — hasSignalOfType() check with overwrite confirmation prompt

6.3 Failure Scenarios

If a user enters wrong data, the application never crashes. Input helpers re-prompt on NumberFormatException. Model constructors throw IllegalArgumentException for constraint violations. FileManager throws typed IOExceptions with line numbers. The main loop's top-level catch ensures the menu always returns after any error.

7. Git Workflow

The project was developed across two days with incremental commits, demonstrating proper version control discipline and staged progression rather than a single bulk upload.

#	Day	Commit Message
1	Day 1	Initialized project structure and created base classes
2	Day 1	Added initial class diagram and architecture design
3	Day 1	Implemented User and DailyRecord classes
4	Day 2	Added BehaviorSignal abstract class and subclasses
5	Day 2	Implemented BehaviorAnalyzer with stress score logic
6	Day 3	Added ReportGenerator for weekly summaries
7	Day 4	Implemented FileManager for saving and loading data
8	Day 4	Created main application flow and user interaction
9	Day 5	Added exception handling for invalid inputs and file errors
10	Day 5	Completed README and documentation

Development Progression

Development began with repository setup and base class design. Model classes (User, DailyRecord, BehaviorSignal hierarchy) were built first, followed by the analysis and reporting layer, then file I/O, and finally the console interface. Each stage was committed with a descriptive message, producing a clean, readable commit history that verifies incremental progress.

8. Conclusion & Future Scope

Conclusion

EmpathAI successfully demonstrates how all eight required Object-Oriented Programming concepts can be applied within a single, cohesive, and purposeful application. The project models users, daily behavioural records, and signal types using a clean class hierarchy. Polymorphic scoring, interface-based reporting, and robust file I/O combine into a system that is both academically rigorous and practically useful.

> The layered architecture — model classes, analysis service, report generator, file manager, and entry point all fully separated — makes the codebase easy to test, extend, and evaluate.

Future Scope

- Database integration using MySQL or SQLite for persistent storage
- Login and authentication system with password protection
- Graphical user interface using Java Swing or JavaFX
- Export wellness reports to PDF
- Email or push notification alerts for consecutive high-risk days
- Thread-based background file saving for non-blocking I/O
- Web API layer so the tracker can be used from a mobile app